

INSTRUCTIVO TÉCNICO

Sistema de Pedidos Asíncronos

API REST · RabbitMQ · PostgreSQL · HAProxy · AWS EC2 · Terraform

Computación en la Nube — Proyecto Final
Pontificia Universidad Javeriana Cali

Sebastián Izquierdo · izqsebas@gmail.com

Mayo 2026

Repositorio: github.com/izqsebas-sys/proyecto-cloud · Swagger UI en producción:
<http://100.54.28.154/docs>

1. Resumen Ejecutivo

Este proyecto implementa un **sistema de procesamiento de pedidos asíncrono** desplegado completamente en AWS usando infraestructura como código (IaC) con Terraform. El sistema combina todas las tecnologías estudiadas durante el semestre — Docker, Python, RabbitMQ, PostgreSQL, HAProxy y AWS — en una solución de producción real.

La arquitectura separa claramente las responsabilidades: la **API** recibe peticiones y las encola, el **Worker** las procesa en segundo plano, y el **balanceador HAProxy** distribuye el tráfico entre dos instancias de la API para garantizar disponibilidad.

HAProxy	1 EC2	Balanceador de carga, único punto de entrada público (puerto 80)
FastAPI (API)	2 EC2	Recibe pedidos, los valida, los guarda en BD y los encola en RabbitMQ
RabbitMQ	1 EC2	Broker de mensajes, desacopla la API del Worker
Worker Python	1 EC2	Consume las colas y ejecuta las operaciones en PostgreSQL
PostgreSQL	1 EC2	Base de datos relacional, almacena pedidos y tareas
AWS SSM	Servicio	Almacén de configuración centralizado para IPs y parámetros

2. Arquitectura del Sistema

El sistema sigue un patrón de **arquitectura basada en eventos**. Las operaciones que pueden tomar tiempo (crear o eliminar un pedido) se ejecutan de forma asíncrona a través de RabbitMQ, mientras que las operaciones rápidas (listar, actualizar) son síncronas.

2.1 Diagrama de Flujo Asíncrono (Creación de Pedido)

```
Cliente → HAProxy (puerto 80) → API-1 o API-2 (puerto 8000)
```

```
API → PostgreSQL: INSERT en tabla 'tasks' (estado: pendiente)
```

```
API → RabbitMQ: publicar en cola 'orders_create'
```

```
API → Cliente: HTTP 202 + task_id
```

```
Worker ← RabbitMQ: consumir mensaje de 'orders_create'
```

```
Worker → PostgreSQL: INSERT en tabla 'orders'
```

```
Worker → PostgreSQL: UPDATE tasks SET estado='completado'
```

```
Cliente → GET /tasks/{task_id} → verifica cuando estado='completado'
```

Este patrón se conoce como **polling de estado**: el cliente pregunta periódicamente hasta que la tarea se completa. Es simple de implementar y no requiere websockets ni notificaciones push.

2.2 Flujo Síncrono (Actualización de Pedido)

```
Cliente → HAProxy → API → PostgreSQL: UPDATE directo → HTTP 200
```

La actualización es síncrona porque es una operación rápida (un UPDATE en BD) que no requiere procesamiento adicional por parte del Worker.

2.3 Comunicación entre Servicios vía SSM

En el Learner Lab de AWS Academy las IPs de las instancias cambian con cada sesión. Para solucionar esto, Terraform guarda las IPs privadas en **AWS SSM Parameter Store** al momento de crear la infraestructura. La API y el Worker las leen al arrancar usando el SDK boto3.

```
/proyecto/postgres_host → IP privada de la instancia PostgreSQL
```

```
/proyecto/rabbitmq_host → IP privada de la instancia RabbitMQ
```

```
/proyecto/haproxy_ip → IP pública del balanceador HAProxy
```

3. Progresión del Curso: De las Prácticas al Proyecto Final

El proyecto final no surgió de la nada. Cada práctica del semestre introdujo una tecnología o concepto que luego se integró en la solución final. La siguiente tabla muestra esa progresión:

Práctica	Tecnología	Qué aprendimos	Uso en el Proyecto
0. Dev Environment	Docker + Dockerfile	Crear imágenes reproducibles sin depender del SO del desarrollador	API y Worker corren en contenedores Docker en cada EC2
Practice 1 HTML + Docker	Contenedor web básico	Empaquetar una aplicación en un contenedor y exponerla en un puerto	Base para entender cómo Docker expone servicios al exterior
Practice 3 Flask + Redis	Flask, Redis, docker-compose	API web con estado persistente, múltiples servicios en red	Flask → FastAPI. Redis (caché) → PostgreSQL (persistencia real)
Practice 4 RabbitMQ	pika, colas AMQP, fanout exchange	Publicar y consumir mensajes, desacoplar productores de consumidores	Patrón exacto usado entre la API (producer) y el Worker (consumer)
Lab 1 AWS Lambda básico	AWS Lambda, AWS CLI, zip deploy	Primer contacto con AWS, credenciales, despliegue serverless	Familiarización con AWS y boto3, base para usar SSM
Lab 3 Lambda + EC2	EC2, Lambda, AWS CLI	Crear instancias EC2 y conectar Lambda con recursos de red	EC2 es el modelo de cómputo del proyecto final (IaaS)
Lab 4 Lambda + S3	S3, Terraform básico, IaC	Infraestructura como código: describir recursos en archivos .tf	Terraform escala al proyecto: 6 EC2 + security groups + SSM con un solo apply
Lab 6 Lambda + DynamoDB	DynamoDB, Lambda, múltiples funciones, simulador IoT	Patrón ingesta + consulta, arquitectura de múltiples componentes	Motivó usar PostgreSQL (relacional) en lugar de DynamoDB para el proyecto

aws-iaas-examples ALB, RabbitMQ+PostgreSQL	ALB, HAProxy, SSM Parameter Store, múltiples EC2	Balanceadores de carga, configuración centralizada con SSM, instalación automatizada	Template directo del proyecto: mismo patrón de install_*.sh y SSM
---	--	---	--

4. Justificación de Tecnologías y Librerías

4.1 Python 3.x — Lenguaje Principal

Python fue el lenguaje usado en todas las prácticas del curso (Lambda, Flask, pika, boto3). Su ecosistema de librerías para nube (boto3), mensajería (pika) y bases de datos (psycopg2) cubre exactamente lo que necesita el proyecto. No fue necesario aprender un lenguaje nuevo.

4.2 FastAPI — Framework Web de la API

Tecnología	Versión / Tier	Justificación
FastAPI	0.115.x	Framework web moderno de Python. Genera automáticamente la documentación Swagger UI (/docs), valida los datos de entrada con Pydantic sin código extra, y es considerablemente más rápido que Flask (usado en Practice 3). La validación automática de tipos previene errores de datos sin escribir código de validación manual.
Uvicorn	ASGI server	Servidor de producción para FastAPI. Es el equivalente a Gunicorn pero para aplicaciones ASGI (asíncronas). Se especifica en el Dockerfile como comando de arranque.
Pydantic	Incluida en FastAPI	Valida automáticamente el JSON recibido contra los modelos OrderCreate y OrderUpdate. Si el cliente envía un campo inválido, FastAPI retorna un error 422 descriptivo sin necesidad de código adicional.

4.3 RabbitMQ + pika — Mensajería Asíncrona

RabbitMQ se introdujo en la **Práctica 4** donde se aprendió a publicar y consumir mensajes con el patrón fanout (broadcast). En el proyecto se usa el patrón más simple de **cola directa** (direct queue): un mensaje va a una cola específica y un solo Worker lo procesa.

¿Por qué RabbitMQ y no SQS de AWS? El Learner Lab tiene permisos limitados para crear servicios adicionales de AWS. Además, RabbitMQ se estudió explícitamente en el curso y su instalación con Docker es directa. En producción real se usaría Amazon SQS por ser administrado y no requerir mantenimiento.

Tecnología	Versión / Tier	Justificación
------------	----------------	---------------

pika	1.3.x	Librería oficial de Python para el protocolo AMQP (el protocolo de RabbitMQ). Usada en Practice 4. Permite publicar mensajes desde la API y consumirlos en el Worker con callbacks.
------	-------	---

4.4 PostgreSQL — Base de Datos Relacional

PostgreSQL se introdujo en los laboratorios de EC2 con el script *ec2-user-data-postgresql.sh* y en los ejemplos de IaaS. Se eligió PostgreSQL sobre otras alternativas por las siguientes razones:

Tecnología	Versión / Tier	Justificación
PostgreSQL	15.x	Base de datos relacional con soporte nativo para JSONB (JSON Binario). El campo 'payload' de los pedidos se almacena como JSONB, lo que permite actualizar campos individuales con el operador ' ' sin reescribir todo el objeto. Esto hace el endpoint PUT eficiente y flexible.
psycopg2	2.9.x	Conector Python para PostgreSQL. Es el estándar de la industria. Usa consultas parametrizadas (%s) que previenen inyección SQL automáticamente.
pgcrypto	Extensión PostgreSQL	Extensión habilitada en el script de instalación para soporte de UUIDs nativos en la base de datos. Las tablas 'tasks' y 'orders' usan UUID como clave primaria en lugar de enteros secuenciales.

¿Por qué PostgreSQL y no DynamoDB? En el Lab 6 se estudió DynamoDB, que es excelente para datos sin esquema fijo y alta escala. Sin embargo, el proyecto tiene relaciones claras entre tablas (tasks ↔ orders mediante foreign key) y necesita transacciones ACID para garantizar consistencia. PostgreSQL es la herramienta correcta para datos estructurados con relaciones.

4.5 HAProxy — Balanceador de Carga

HAProxy se estudió en la carpeta *balancers/* y en los ejemplos *aws-iaas-examples/aws-iaas-alb/*. Se eligió HAProxy sobre el Application Load Balancer (ALB) de AWS porque:

Tecnología	Versión / Tier	Justificación
HAProxy	2.8.x (Docker)	Balanceador de carga open source. Distribuye las peticiones entre API-1 y API-2 con el algoritmo round-robin (alterna entre servidores). Si una API falla, HAProxy deja de enviarle tráfico automáticamente. El Learner Lab tiene permisos limitados para crear ALBs pagos, mientras que HAProxy corre gratis en una EC2 t3.micro.

4.6 Docker — Contenerización de la API y el Worker

Tecnología	Versión / Tier	Justificación
Docker	24.x	La API y el Worker corren en contenedores Docker dentro de sus EC2. Esto garantiza que el entorno de ejecución es idéntico al de desarrollo (el mismo Dockerfile, las mismas dependencias). Sin Docker, habría que instalar Python, pip y cada librería manualmente en cada instancia.
Dockerfile	Multi-stage requerido	no Cada componente (api/, worker/) tiene su propio Dockerfile que parte de python:3.11-slim, instala requirements.txt y define el comando de arranque. python:3.11-slim se eligió sobre python:3.11 porque ocupa menos espacio en disco.

4.7 Terraform — Infraestructura como Código (IaC)

Terraform se introdujo en el **Lab 4 (Lambda + S3)** donde se aprendió a definir recursos en archivos .tf y aplicarlos con *terraform apply*. En el proyecto, Terraform gestiona 9 recursos de AWS con un solo comando:

```
terraform apply -auto-approve
```

Tecnología	Versión / Tier	Justificación
Terraform	≥ 1.11 / OpenTofu	Define los 6 EC2, los 5 security groups y los 3 parámetros SSM como código. Permite destruir y recrear toda la infraestructura en minutos. Compatible con OpenTofu (fork open source de Terraform), usado en los laboratorios del curso.

templatefile()	Función de Terraform	Inyecta variables (IP de la API, contraseña de BD, URL del repo) en los scripts bash de instalación. Evita hardcodear valores que cambian entre sesiones del Learner Lab.
AWS SSM Parameter Store	Servicio AWS	Terraform guarda las IPs privadas de PostgreSQL y RabbitMQ en SSM después de crear las instancias. La API y el Worker las leen con boto3 al arrancar. Soluciona el problema de que las IPs cambian con cada sesión del Learner Lab.

4.8 boto3 — SDK de AWS para Python

Tecnología	Versión / Tier	Justificación
boto3	1.34.x	SDK oficial de AWS para Python. Se usa para leer los parámetros de SSM desde la API, el Worker y el Producer. Sin boto3 habríamos tenido que hacer llamadas HTTP manuales a la API de SSM con autenticación compleja. boto3 maneja automáticamente la autenticación con los credenciales del perfil de instancia IAM (LabInstanceProfile).

4.9 pytest y ruff — Calidad de Código

Tecnología	Versión / Tier	Justificación
pytest	8.3.x	Framework de pruebas unitarias. Los 17 tests cubren todos los endpoints de la API usando mocks para PostgreSQL y RabbitMQ. Esto permite verificar la lógica de la API sin necesidad de tener las bases de datos corriendo.
ruff	0.x	Linter y formateador de Python extremadamente rápido (escrito en Rust). Detecta imports sin usar, imports desordenados y problemas de estilo. Se configura en pyproject.toml para aplicar las mismas reglas a todos los módulos del proyecto.
unittest.mock	Librería estándar Python	Permite reemplazar las conexiones reales a PostgreSQL y RabbitMQ con objetos falsos (MagicMock) durante las pruebas. Así los tests son rápidos y no dependen de infraestructura externa.

5. Estructura del Repositorio

```
proyecto/
├── api/
│   ├── main.py ← API REST (FastAPI): endpoints, validación, lógica
│   ├── requirements.txt ← Dependencias Python de la API
│   ├── Dockerfile ← Imagen Docker de la API
│   └── tests/
│       ├── conftest.py ← Configuración de fixtures para pytest
│       └── test_main.py ← 17 tests unitarios de todos los endpoints
├── worker/
│   ├── worker.py ← Consumidor RabbitMQ: crea y elimina pedidos en BD
│   ├── requirements.txt ← Dependencias Python del Worker
│   └── Dockerfile ← Imagen Docker del Worker
├── producer/
│   ├── producer.py ← Generador sintético de pedidos para pruebas de carga
│   └── requirements.txt ← Dependencias del Producer
├── terraform/
│   ├── main.tf ← 6 EC2 + 3 parámetros SSM
│   ├── security_groups.tf ← Reglas de firewall por servicio
│   ├── variables.tf ← VPC, subnet, AMI, key pair, repo URL
│   ├── providers.tf ← Proveedor AWS, región us-east-1
│   ├── outputs.tf ← IPs y URLs que se muestran al hacer apply
│   ├── install_postgres.sh ← Instala PostgreSQL, crea BD y tablas
│   ├── install_rabbitmq.sh ← Instala RabbitMQ con Docker
│   ├── install_api.sh ← Clona repo, construye y corre contenedor API
│   ├── install_haproxy.sh ← Instala HAProxy con IPs de las APIs
│   ├── install_worker.sh ← Clona repo, construye y corre contenedor Worker
│   └── Makefile ← Atajos: make apply, make destroy
├── pyproject.toml ← Configuración de ruff (linter)
├── .gitignore ← Excluye __pycache__, .env, .claude/
└── README.md ← Instrucciones de despliegue y uso
```

6. Esquema de Base de Datos PostgreSQL

La base de datos **ordersdb** tiene dos tablas relacionadas. La relación entre ellas permite rastrear exactamente qué tarea originó cada pedido:

task_id	UUID (PK)	Identificador único de la tarea asíncrona
status	VARCHAR(20)	Estado actual: 'pendiente' → 'completado'
created_at	TIMESTAMPTZ	Fecha y hora de creación (con zona horaria UTC)
updated_at	TIMESTAMPTZ	Fecha de última actualización (lo actualiza el Worker)

order_id	UUID (PK)	Identificador único del pedido, generado por el Worker
task_id	UUID (FK → tasks)	Referencia a la tarea que originó este pedido
payload	JSONB	Datos del pedido (description, quantity, product). JSONB permite actualizar campos individuales con el operador ' '
created_at	TIMESTAMPTZ	Fecha de creación del pedido

7. Endpoints de la API REST

GET	/health	Síncrono	Verificación de salud. HAProxy lo usa para saber si la API está activa	200
POST	/orders	Asíncrono	Crea un pedido. Retorna task_id. El Worker procesa en segundo plano	202
GET	/orders	Síncrono	Lista todos los pedidos ordenados del más reciente al más antiguo	200
PUT	/orders/{id}	Síncrono	Actualiza campos del pedido usando merge JSONB	200
DELETE	/orders/{id}	Asíncrono	Elimina un pedido. Retorna task_id. El Worker ejecuta el DELETE	202
GET	/tasks/{id}	Síncrono	Consulta el estado de una tarea (pendiente → completado)	200/404

8. Guía Paso a Paso: Despliegue en AWS Learner Lab

Paso 1 — Obtener credenciales AWS del Learner Lab

En la consola del Learner Lab: **AWS Details** → **Show** → **Copiar las 3 variables**

```
export AWS_ACCESS_KEY_ID=ASIA...
```

```
export AWS_SECRET_ACCESS_KEY=...
```

```
export AWS_SESSION_TOKEN=...
```

Paso 2 — Actualizar VPC y Subnet en variables.tf

Las IDs de VPC y subnet cambian con cada sesión del Learner Lab. Obtenerlas con:

```
aws ec2 describe-vpcs --region us-east-1 --query 'Vpcs[*].{VpcId:VpcId}' --output table
```

```
aws ec2 describe-subnets --region us-east-1 --filters 'Name=vpc-id,Values=VPC_ID'
```

Actualizar *variables.tf*: campos **vpc_id** y **subnet_id**.

Paso 3 — Desplegar toda la infraestructura

```
cd proyecto/terraform
```

```
terraform init # Descarga el proveedor AWS
```

```
terraform apply -auto-approve # Crea los 9 recursos en ~30 segundos
```

Al terminar, Terraform muestra las IPs de todas las instancias y la URL del Swagger.

Paso 4 — Verificar que la API está lista (~3-5 minutos)

Los scripts de instalación (user_data) tardan varios minutos en ejecutarse. Esperar hasta que el health check responda:

```
curl http://HAPROXY_IP/health  
  
# Respuesta esperada: {"estado": "saludable"}
```

Paso 5 — Probar todos los endpoints

```
# Crear pedido (asíncrono – retorna 202)  
curl -X POST http://HAPROXY_IP/orders \  
-H 'Content-Type: application/json' \  
-d '{"description": "Mi pedido", "quantity": 3, "product": "Widget A"}'  
  
# Verificar estado de la tarea  
curl http://HAPROXY_IP/tasks/TASK_ID  
  
# Listar pedidos  
curl http://HAPROXY_IP/orders  
  
# Destruir infraestructura (para no gastar créditos del Learner Lab)  
terraform destroy -auto-approve
```

9. Decisiones de Diseño: ¿Por qué IaaS y no Serverless?

Durante el curso se trabajaron dos modelos de cómputo en AWS:

Control del SO	Sin acceso — AWS gestiona todo	Control total: instalar cualquier software
Servicios de terceros	Limitado a servicios AWS	Cualquier software: RabbitMQ, HAProxy, PostgreSQL
Servidor largo corriendo	No aplica (stateless, efímero)	El Worker necesita correr indefinidamente escuchando colas
Complejidad de red	Lambda gestiona la red automáticamente	Requiere configurar VPC, security groups, subnets
Costo en lab	Créditos por ejecución	t3.micro casi gratuito en Learner Lab
Aprendizaje	Oculto la infraestructura subyacente	Expone todos los conceptos de red y sistemas operativos

La decisión de usar IaaS fue deliberada: el objetivo era demostrar el dominio de todos los conceptos vistos en el curso (redes, seguridad, contenedores, mensajería, bases de datos) en una solución integrada. Lambda habría ocultado esa complejidad.

10. Resumen de Pruebas

10.1 Pruebas Unitarias (pytest — 17 tests, 100% pasando)

Test	Qué verifica
test_health	GET /health retorna {"estado": "saludable"}
test_create_order_returns_202	POST /orders retorna HTTP 202 con task_id y estado='pendiente'
test_create_order_inserts_task_and_publishes	La API inserta en BD Y publica en RabbitMQ
test_create_order_db_error_returns_500	Error de BD retorna HTTP 500
test_get_task_found	Tarea existente retorna sus datos con estado
test_get_task_not_found	UUID válido pero inexistente retorna 404
test_get_task_invalid_uuid_returns_404	UUID con formato inválido retorna 404 (no 500)
test_get_task_invalid_uuid_message	Mensaje de error correcto en español y género
test_get_task_db_error_returns_500	Error de BD en consulta retorna HTTP 500
test_get_orders_empty	Lista vacía retorna [] con HTTP 200
test_get_orders_with_data	Lista con pedidos retorna los campos en español
test_update_order_success	PUT actualiza y retorna pedido_id y estado='actualizado'
test_update_order_not_found	PUT con pedido inexistente retorna 404
test_update_order_invalid_uuid_returns_404	UUID inválido en PUT retorna 404
test_delete_order_returns_202	DELETE retorna 202 con task_id
test_delete_order_not_found	DELETE con pedido inexistente retorna 404
test_delete_order_invalid_uuid_returns_404	UUID inválido en DELETE retorna 404

10.2 Pruebas de Integración en Producción (AWS)

Después del despliegue en AWS, se verificaron manualmente todos los flujos:

- ✓ GET /health → {"estado": "saludable"} (HTTP 200)
- ✓ POST /orders → task_id generado (HTTP 202)
- ✓ Flujo asíncrono: tarea pasa a 'completado' en < 1 segundo
- ✓ GET /orders → lista el pedido creado con sus datos
- ✓ PUT /orders/{id} → actualiza quantity y description en JSONB
- ✓ DELETE /orders/{id} → tarea completada y lista vacía

✓ UUID inválido → HTTP 404 con mensaje en español (no HTTP 500)

✓ Load balancer → HAProxy distribuye entre API-1 y API-2

10.3 Análisis Estático de Código (ruff)

```
ruff check api/ worker/ producer/
```

```
# Resultado: 0 errores (7 corregidos automáticamente con --fix)
```

Los errores corregidos fueron: imports sin usar, imports desordenados. ruff es más rápido que flake8+black combinados y se configuró en pyproject.toml.

Conclusiones

Este proyecto integra de forma cohesiva todas las tecnologías estudiadas durante el semestre de Computación en la Nube. Cada componente tiene una justificación técnica clara y está conectado directamente con alguna práctica del curso.

La arquitectura asíncrona con RabbitMQ demuestra un patrón real de la industria: desacoplar la recepción de peticiones de su procesamiento, permitiendo que el sistema escale horizontalmente añadiendo más Workers o más instancias de la API detrás del balanceador sin cambiar el código.

El uso de Terraform garantiza que la infraestructura es reproducible: cualquier persona con acceso al repositorio y credenciales de AWS puede recrear el sistema completo en menos de 5 minutos con un solo comando.

La cobertura de tests del 100% de los endpoints y el análisis estático con ruff aseguran que el código es correcto y mantenible, siguiendo estándares de calidad de la industria.

Repositorio: github.com/izqsebas-sys/proyecto-cloud
Generado automáticamente con reportlab · Mayo 2026